Concurrency in Linux Device Drivers — Practical Guide

How to avoid races, deadlocks, and ordering bugs across process context, IRQs, bottom halves, and DMA.

1) Where concurrency comes from

- Multiple user threads calling file operations concurrently (open/read/write/ioctl/mmap).
- Interrupt handlers running asynchronously to process context.
- Bottom halves (softirq/tasklet/NAPI) and workqueues executing in parallel with other contexts.
- Multiple CPUs touching shared state (true parallelism on SMP).
- DMA engines modifying memory while the CPU reads/writes it.

2) Choose synchronization by context (sleeping vs atomic)

The first question is: can this code path sleep? In hard IRQ/atomic context you must not sleep → use spinlocks, atomics, or preallocated resources. In process context you can sleep → use mutexes, completions, and wait queues.

3) Core primitives and when to use them

3.1 Spinlocks (safe in IRQ/atomic)

Use spinlocks for short critical sections shared with IRQ handlers or other atomic contexts. Prefer irqsave/irqrestore in process context if the same lock is taken in IRQ.

```
/* Shared with IRQ: process context uses irqsave */
spinlock_t lock;
unsigned long flags;

/* Process context */
spin_lock_irqsave(&lock, flags);
/* critical section */
spin_unlock_irqrestore(&lock, flags);

/* Hard IRQ handler */
irqreturn_t my_irq(int irq, void *data)
{
    struct mydev *d = data;
    spin_lock(&d->lock);
    /* critical section */
    spin_unlock(&d->lock);
    return IRQ_HANDLED;
}
```

3.2 Mutexes (sleepable)

Use mutexes to protect longer operations in process context (e.g., configuration ioctls, slow I/O paths). Never use a mutex in hard IRQ.

```
struct mutex cfg_lock;

long my_ioctl(struct file *f, unsigned int cmd, unsigned long arg)
{
```

```
    long ret;
    mutex_lock(&cfg_lock);
    /* may sleep: copy_from_user, allocate memory, talk to firmware */
    ret = do_config(cmd, arg);
    mutex_unlock(&cfg_lock);
    return ret;
}
```

3.3 Atomics and bitops (small counters/flags)

Use atomic operations for simple counters and state flags. For multi-field invariants, use locks.

```
atomic_t inflight = ATOMIC_INIT(0);

atomic_inc(&inflight);
/* ... */
if (atomic_dec_and_test(&inflight))
    wake_up(&wq);
```

3.4 Completions and wait queues (sleep until event)

Use completions when one thread waits for a specific event (often signaled from IRQ). Use wait queues when threads wait for a condition.

```
DECLARE_COMPLETION(cmd_done);

/* submit command */
reinit_completion(&cmd_done);
start_hw_command(d);

/* wait (process context) */
if (!wait_for_completion_timeout(&cmd_done, msecs_to_jiffies(500)))
    return -ETIMEDOUT;

/* signal from IRQ */
complete(&cmd_done);
```

4) Top-half / bottom-half design patterns

Keep hard IRQ handlers minimal: acknowledge/disable interrupts, snapshot status, queue work, and return. Defer heavy work to a threaded IRQ, NAPI poll, or workqueue.

4.1 Workqueues (sleepable deferred work)

```
/* IRQ queues work; workqueue runs in process context (can sleep) */
static void my_workfn(struct work_struct *w)
{
    struct mydev *d = container_of(w, struct mydev, work);
    mutex_lock(&d->cfg_lock);
    handle_event_slow(d);
    mutex_unlock(&d->cfg_lock);
}

irqreturn_t my_irq(int irq, void *data)
{
    struct mydev *d = data;
    /* ack irq, read status */
    schedule_work(&d->work);
    return IRQ_HANDLED;
}
```

4.2 Threaded IRQs (preferred when you need sleeping)

Threaded IRQs split into a fast top-half and a kernel thread handler that may sleep. Useful for slow I/O or register sequences.

```
/* request_threaded_irq(dev, irq, top, thread, IRQF_ONESHOT, name, d); */
irqreturn_t top(int irq, void *data)
{
    /* quick: mask/ack */
    return IRQ_WAKE_THREAD;
}

irqreturn_t thread(int irq, void *data)
{
    struct mydev *d = data;
    mutex_lock(&d->cfg_lock);
    handle_event_slow(d);
    mutex_unlock(&d->cfg_lock);
    return IRQ_HANDLED;
}
```

5) DMA concurrency and cache coherency

DMA runs concurrently with the CPU. The DMA mapping API (dma_map_*/dma_unmap_*) and sync calls ensure correct cache visibility on non-coherent systems and correctness with IOMMUs.

```
/* Device writes into buffer (FROM_DEVICE). CPU must sync before reading. */
dma_addr_t dma = dma_map_single(dev, buf, len, DMA_FROM_DEVICE);
if (dma_mapping_error(dev, dma))
    return -EIO;

submit_rx_to_device(dma);

/* later, after device signals completion */
dma_sync_single_for_cpu(dev, dma, len, DMA_FROM_DEVICE);
consume_packet(buf, len);
```

```
dma_sync_single_for_device(dev, dma, len, DMA_FROM_DEVICE);
dma_unmap_single(dev, dma, len, DMA_FROM_DEVICE);
```

6) Memory ordering, MMIO, and barriers

Even with locks, you may need explicit ordering between normal memory and MMIO (posted writes). Use readl/writel and appropriate barriers (wmb/rmb/mb) according to the subsystem and device requirements.

```
/* Ensure descriptors are visible before ringing the doorbell */
write_desc_ring(d);
dma_wmb();              /* order CPU writes before MMIO notify */
writel(DOORBELL, d->mmio + REG_DB);

/* If you need to ensure MMIO write reaches device before continuing */
readl(d->mmio + REG_DB); /* posting read as a flush on some buses */
```

7) Deadlock avoidance rules

- Keep a strict lock ordering (e.g., cfg_lock → queue_lock → stats_lock).
- Never sleep while holding a spinlock (no mutex_lock(), no copy_to/from_user(), no blocking I/O).
- Avoid taking mutexes from hard IRQ handlers; use threaded IRQs or workqueues.
- Minimize lock hold time; split long operations into stages.
- Use lockdep during development to detect inversions and unsafe contexts.

8) Mini driver skeleton (char device) with safe concurrency

This skeleton shows a common pattern: a mutex protects configuration and slow paths; a spinlock protects a ring buffer shared between IRQ and read(); a wait queue blocks readers until data arrives.

```
struct mydev {
    struct device *dev;
    void __iomem *mmio;

    /* Slow-path configuration */
    struct mutex cfg_lock;

    /* Shared with IRQ: ring buffer */
    spinlock_t q_lock;
    u8 *ring;
    size_t head, tail, ring_sz;

    wait_queue_head_t read_wq;

    /* Deferred work */
    struct work_struct work;
};

static inline bool ring_empty(struct mydev *d)
{
    return d->head == d->tail;
}

irqreturn_t my_irq(int irq, void *data)
{
    struct mydev *d = data;
    unsigned long flags;

    /* ack/clear irq in HW */

    spin_lock_irqsave(&d->q_lock, flags);
    /* push bytes into ring, update head */
    spin_unlock_irqrestore(&d->q_lock, flags);

    wake_up_interruptible(&d->read_wq);
    return IRQ_HANDLED;
}

ssize_t my_read(struct file *f, char __user *ubuf, size_t len, loff_t *off)
{
    struct mydev *d = f->private_data;
    unsigned long flags;
    ssize_t ret;

    /* Sleep until data arrives */
    if (wait_event_interruptible(d->read_wq, !ring_empty(d)))
        return -ERESTARTSYS;

    spin_lock_irqsave(&d->q_lock, flags);
    /* pop from ring into temporary buffer; advance tail */
    spin_unlock_irqrestore(&d->q_lock, flags);

    /* copy_to_user may sleep -> do it outside spinlock */
    ret = copy_to_user(ubuf, kbuf, n) ? -EFAULT : n;
    return ret;
}
```

```
long my_ioctl(struct file *f, unsigned int cmd, unsigned long arg)
{
    struct mydev *d = f->private_data;
    long ret;

    mutex_lock(&d->cfg_lock);
    ret = do_config(d, cmd, arg);
    mutex_unlock(&d->cfg_lock);
    return ret;
}
```

9) Debugging & validation

Enable lock debugging (lockdep) and memory debugging (KASAN/KFENCE) while developing. Use tracepoints/ftrace to confirm ordering and timing. Concurrency bugs are often timing-sensitive, so stress tests (multiple threads + IRQ load) are essential.

Generated by M365 Copilot (guide + examples).